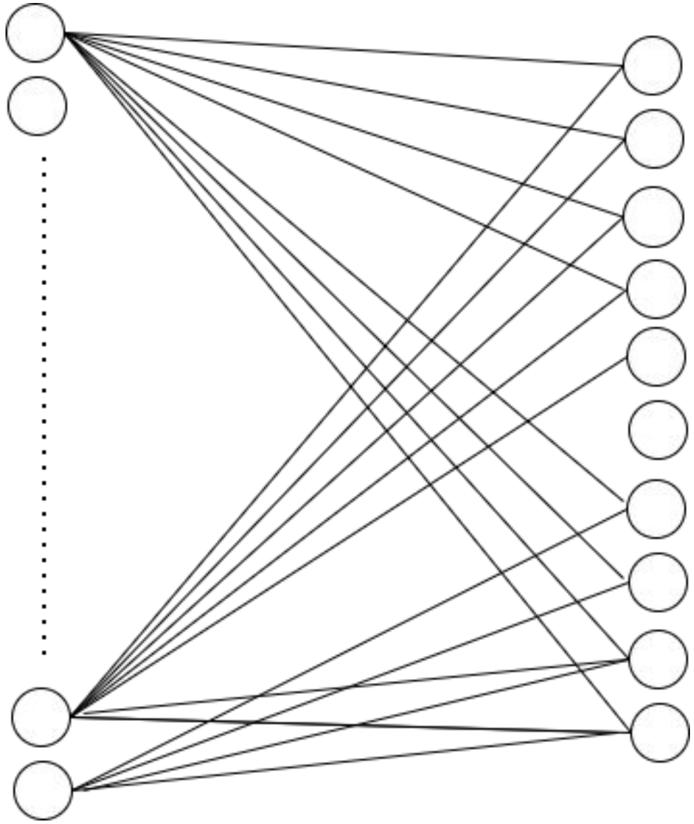
2T0C Memory Cell Array for In-Memory MAC Operations

Mir Maiti
Vishal Hegde

Neural Network Architecture



Design Data Path

Input Stage
file_to_bus (Verilog) - Loads weights and voltages from .mem files

Conversion Stage
dac_3bit (Weight to Voltage) + dac_8bit (Input to Voltage)

Compute Stage
2T0C Cell Array (Verilog-A) - 280 cells performing I = Gm × W × X

Output Stage
ccvs + ADC (Current to Voltage to Digital)

MAC Equation used :

$I_{out} = G_m \times (V_{weight} - V_{th}) \times V_{input}$

Simulation Workflow :

- **Design Entry** → Schematic Capture (Virtuoso Schematic Editor)
- **Compilation** → xrun + xmelab (Verilog/Verilog-A compile)
- **AMS Engine** → Xcelium + Spectre (Co-simulation)
- **Analysis** → ViVA / SimVision (Waveform viewer)

This is the basic overview, since we haven't completed the simulation part, the finer details such as simulation time can be changed.

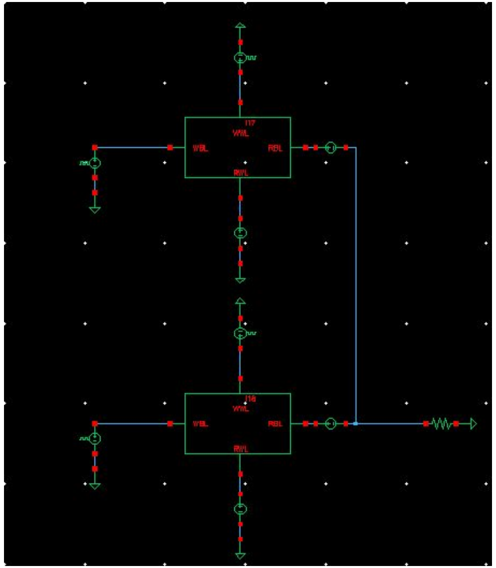
Simulation Methodology

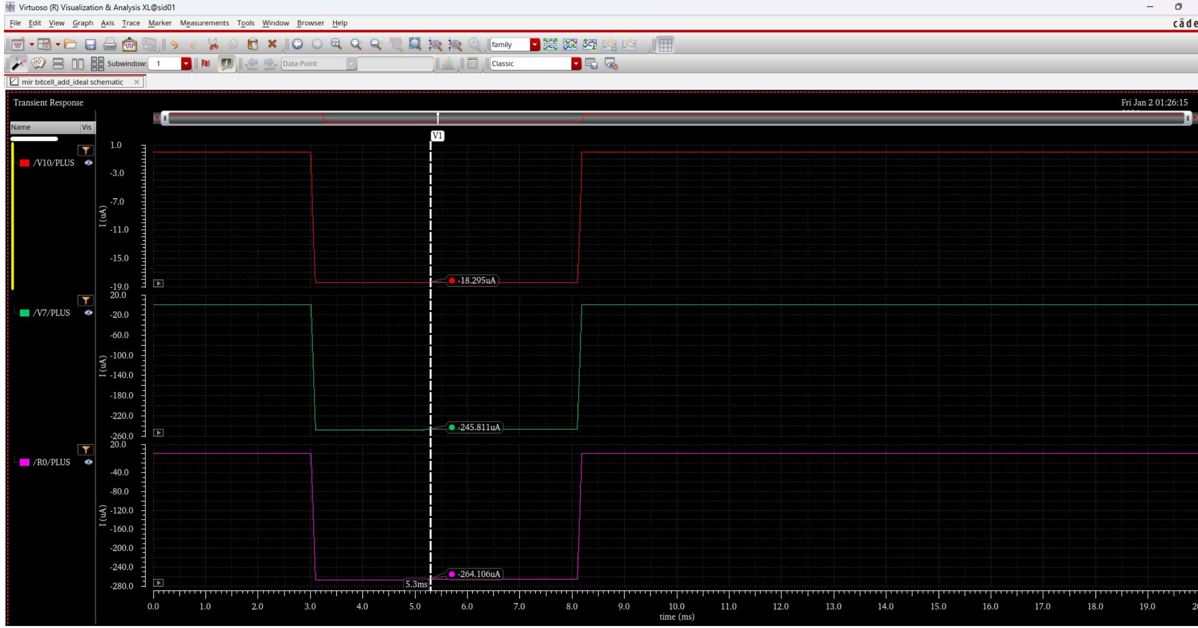
Phase 1: Weight Programming	**Phase 2: MAC Computation**	**Phase 3: Output Readout**
Load .mem files via $readmemh	Apply input voltage (Vds)	CCVS converts I → V
Convert to analog (DAC)	Current = f(Weight, Input)	ADC digitizes output
Store on gate capacitance	Column currents accumulate	Compare with expected

Simulation of Two Memory Cells (To verify building block)

Directory : /sid01/PROJ/TFT/CDS/mir/bitcell_add_ideal

We performed addition operation using two 2T0C cells. Their outputs currents get summed as shown below:





Note: This schematic works when we use "_3bitcell" too, since it is purely analog.

Note

- Since mixed signal simulation(ams) is not supported by the current license we have, the 2T0C cell has been simulation using verilogA, so that we can use spectre for simulation.
- If that problem is resolved, we can replace the file "_3bitcell_digital" with a real version "_3bitcell". (We have made a schematic for this, but it doesn't work when we introduce verilog blocks in the testbench)

Directory of main project: /sid01/PROJ/TFT/CDS/IGZO_fully_connected

- All the modules needed for the final testbench is in this directory. You can open them from the library manager.
- The **important modules** are :
  1. `_3bitcell_digital` : It's the verilog-A model of the 2T0C cell
  2. `analog_NN_column_ideal` : It contains one column with 28 cells
  3. `analog_NN_ideal` : It contains 10 columns
  4. `final_tb_ideal` : It contains the final testbench with all blocks joined to form a neural network

- Integrated supporting blocks:
  - `file_to_bus` (Verilog): Loads weights and input voltages from memory files
  - `dac_3bit_ideal` (Verilog-A): 280 instances for weight conversion
  - `dac_8bit_ideal` (Verilog-A): 280 instances for input conversion
  - `ccvs_hdl` (Verilog-A): Current-to-voltage conversion (rm=3000)
  - `adc_8bit_ideal` (Verilog-A): 10 instances for output digitization
  - `Aggregator`, `stride_split` (SystemVerilog): Data routing

Input Data Sources

Weight Matrix Files (`oink_column_0.txt` to `oink_column_9.txt`)

These files represent the pre-trained weights of the neural network.

- **Content:** Each file contains quantized 3-bit weight values.
- **Mapping:** Each "column" file corresponds to a specific vertical column in the memory array.
- **Function:** During the initial "Write" phase, the testbench reads these values and applies them to the **WBL (Write Bitline)** of the memory cells while **WWL (Write Wordline)** is held high.
- **Quantization:** The values are mapped to an analog voltage range of **2.1V to 2.8V** to represent the 8 possible 3-bit states.

Image Data File (mnist_input_data_hex.txt)

This file contains the actual handwritten digit data used for inference.

- **Content:** Hexadecimal representation of MNIST image pixels.
- **Function:** This data is fed into the **RWL (Read Wordline)** of the memory units.
- **Signal Range:** Values are translated into voltages between **0V and 1.4V**.
- MAC Operation: The interaction between these input voltages and the stored gate voltages (weights) produces a proportional current on the RBL (Read Bitline), effectively calculating:

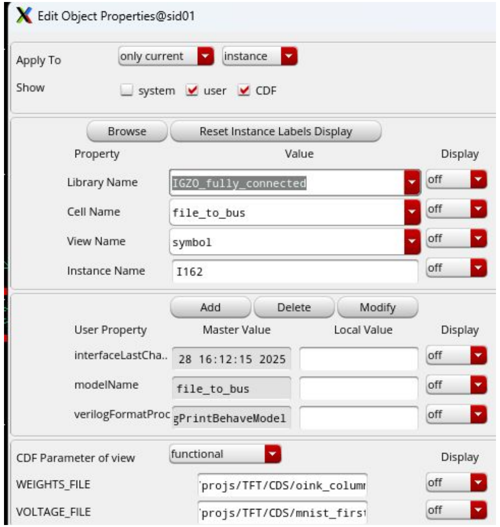
Output Signals

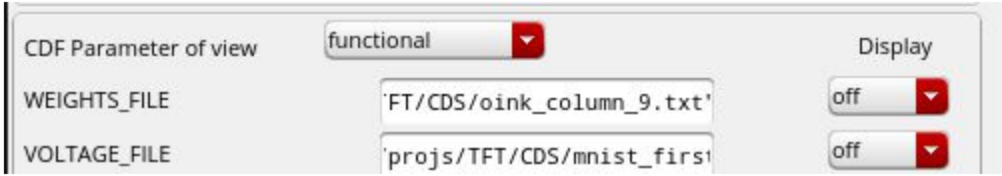
The outputs are represented by the buses on the right side of the aggregator schematic, currently terminated by 1kΩ load resistors for measurement.

Output Bus	Description
currentSums<319:0>	The running total of the MAC operations for the current mini-batch.
max_index<3:0>	**The Final Result:** The 4-bit index (0-9) representing the predicted MNIST digit.
max_prod_value<31:0>	The maximum dot-product value found during the current classification cycle.
done	A flag indicating that the mini-batch has been fully processed and the output is valid.

How to change the input weights and mnist_input data

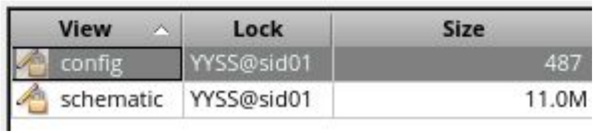
1. The file_to_bus takes in input from the local directory. From where the virtuoso application was launched.
2. To edit the file names, go to properties of the file_to_bus instance -> CDS Parameter of view -> Change the “WEIGHTS_FILE” and “VOLTAGE_FILE”



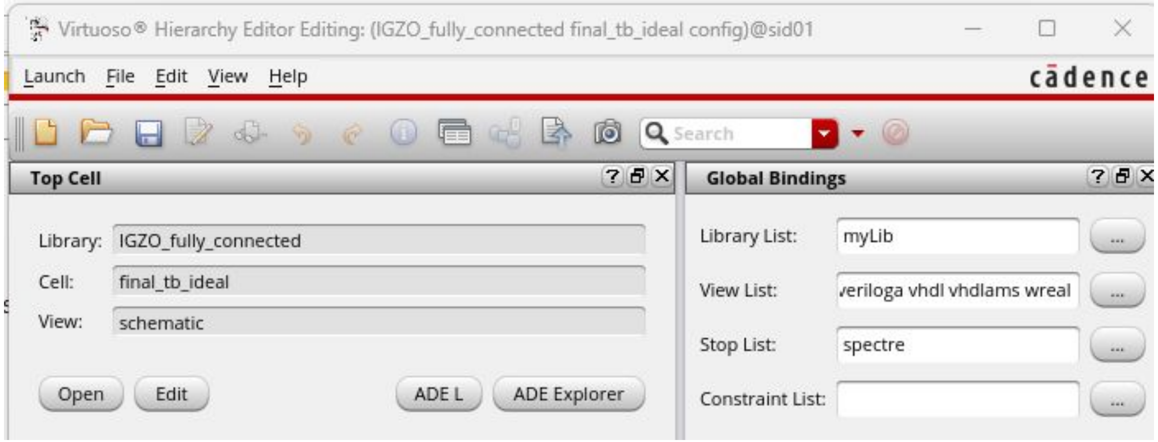


Steps to simulate:

1. These are the views required to do simulation -

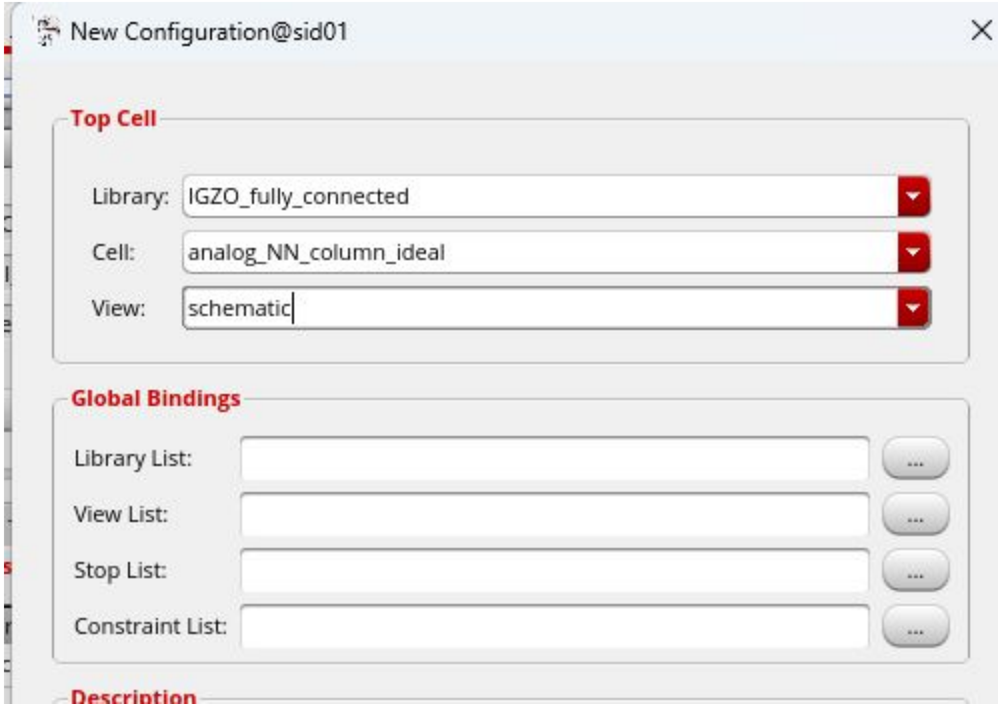


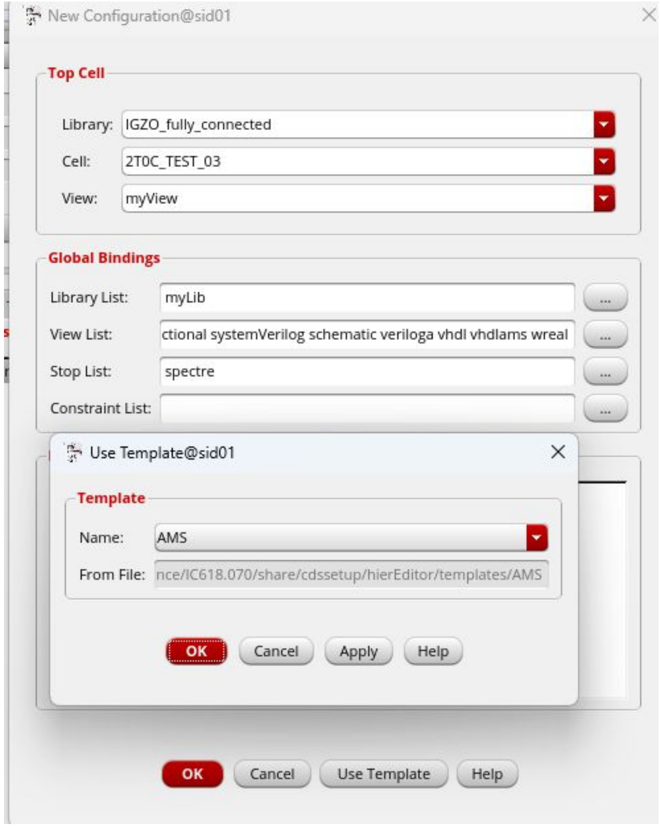
2. Once they are ready, go to Hierarchy Editor by double clicking on the config view shown above. You should get a window like this -



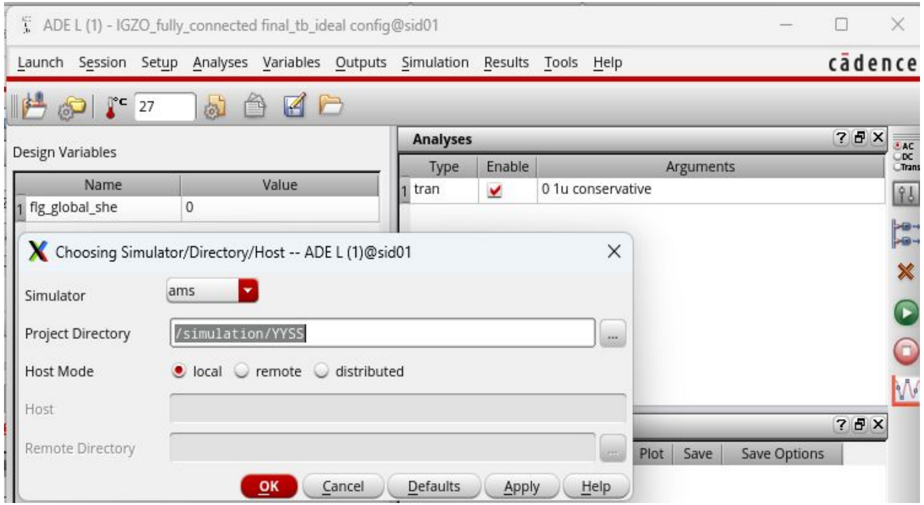
3. In the New Configuration Window, click on Use Template and select “AMS”.

4. Change the Top Cell View from “myView” to “schematic”. Save the config file.

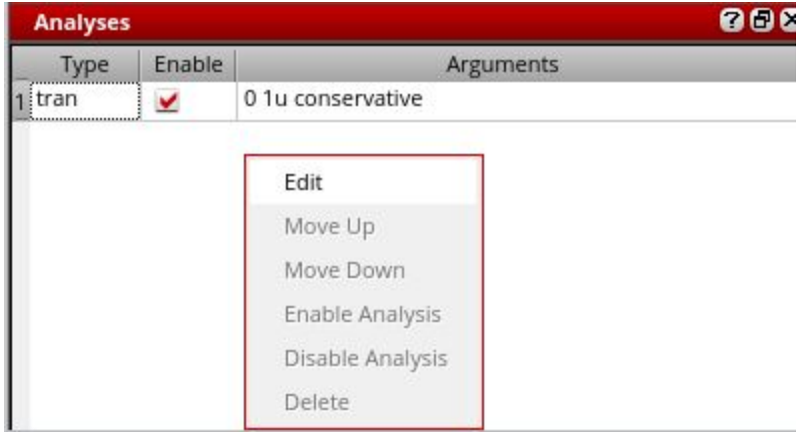




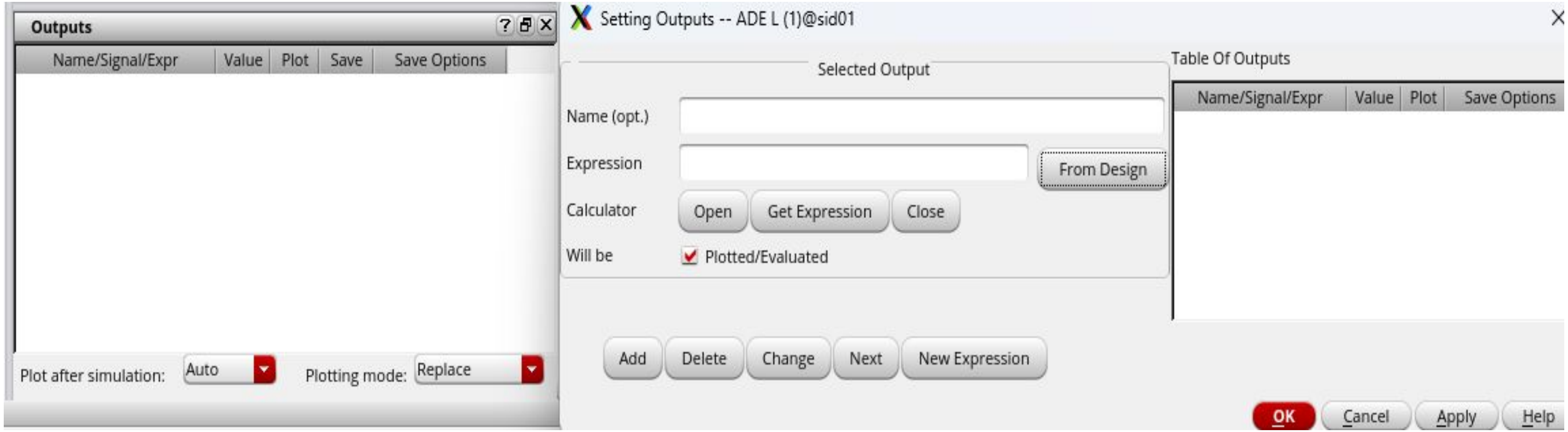
5. Go to ADE L -> Setup -> Simulator/ Directory/Host -> ams



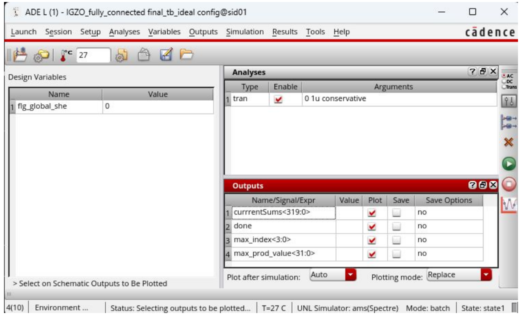
6. Choose the type of analysis you want in the analyses window. Right click to see the options.



7. Select the outputs you want to plot by right click on “Outputs” window and manually choosing from design.

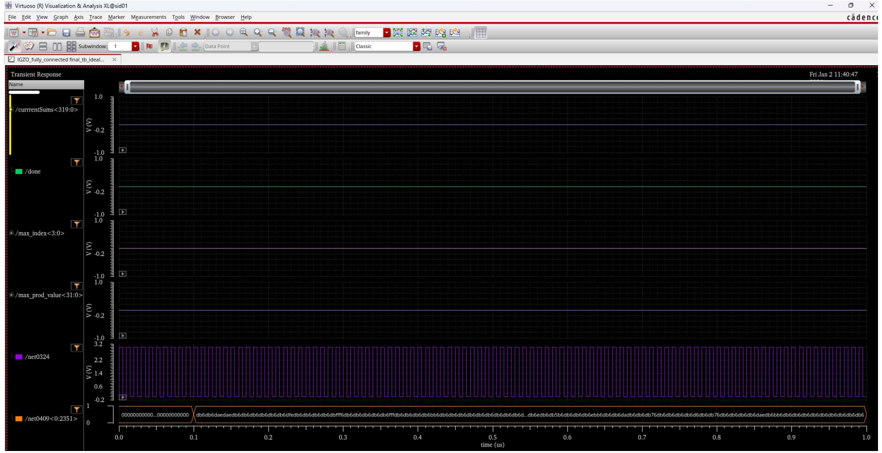


8. Once all the outputs to be plotted are selected, it should look something like this-



9. Click the green run button on the rightmost window to start the simulation process.

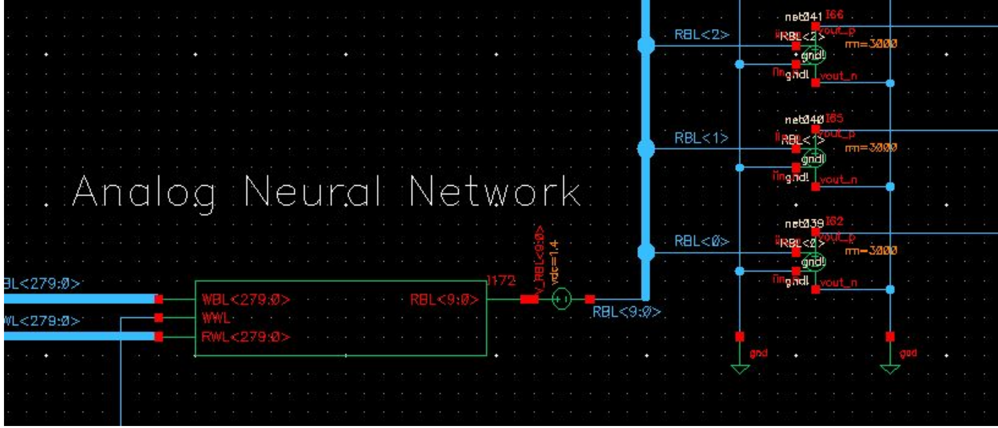
10. Once simulation is done, Virtuoso ® Visualization & Analysis XL will pop up in a new window, showing you the plots.



Limitations:

1. The “rm” value of the ccvs needs to be adjusted to ensure that the output voltage always remains in the range of the ADC.
2. There are some unknown problems in the testbench which need to be debugged.

Due to these issues, we are unable to receive proper data output.



Add vivado testbench for the digital blocks